Business case: Bots used up the daily limit. Real users could not work.



Nikita, DevOps engineer
Real case from my work. Client name hidden.

Business	An online service. Users upload shop receipts and check them.
Risk	The service has a fixed daily limit of receipt checks. Bots used it up. Real users got errors.
My work	Safe Docker setup, closed internal network, bot protection, test copy, alerts.
Result	The bot attacks on the paid limit stopped. The team can rebuild the server from code.

The business risk

- The service checks each receipt through an external provider. The provider gives a fixed number of checks per day.
- Bots sent many false requests. They used up the daily limit. Real users then could not check receipts.
- Each attack made the service useless for real users until the next day.
- The server setup was done by hand. A second copy for safe tests did not exist.

What I did

- I recorded each request with its source. This showed how the bots worked.
- I added a captcha, a limit on requests per user and a queue. These stop bot traffic before it reaches the external check.
- I added automatic blocks for sources that send too many requests.
- I packed each part of the service in Docker and put all parts in one closed internal network. Only the website is open to the internet.
- I wrote the server setup as code and made a test copy. The team tests changes before the release.
- I connected the service to monitoring. The team gets an alert when the site is not available.

Bots and users
many false requests

Bot protection
captcha, limits, queue

The app
closed internal network

Daily limit
used by real users only

Result

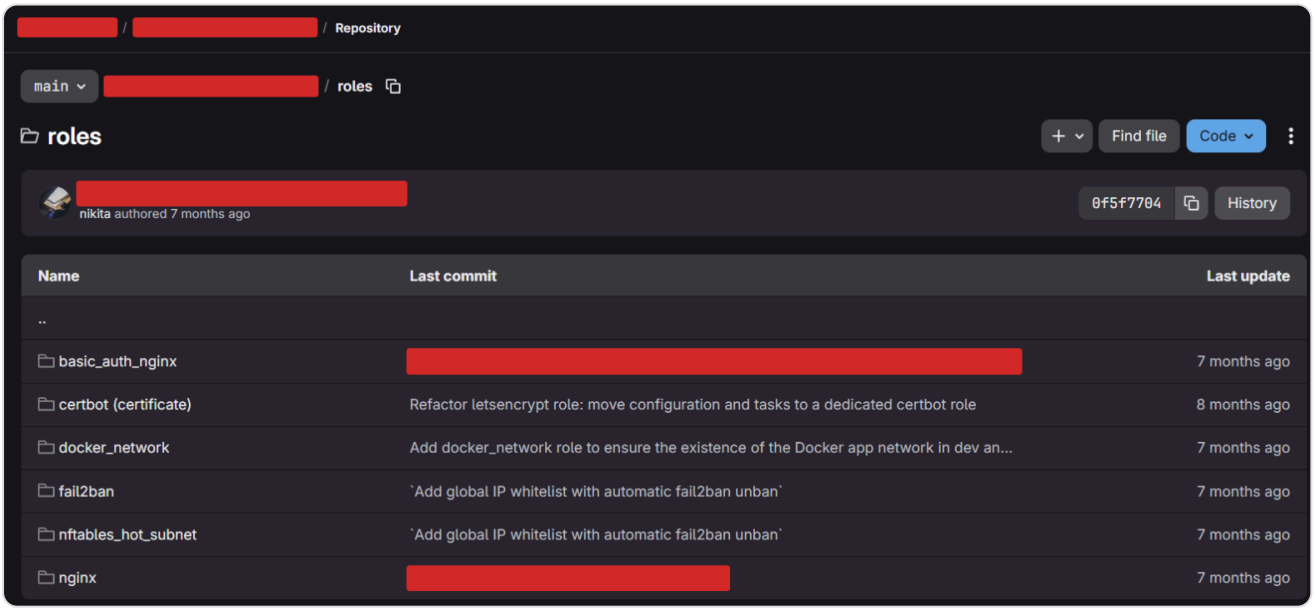
- The bot attacks on the paid limit stopped after the changes.
- Real users can check receipts again.
- The database and internal parts are not open to the internet.
- The team can build a new server or a test copy from saved setup code.

More results from my work

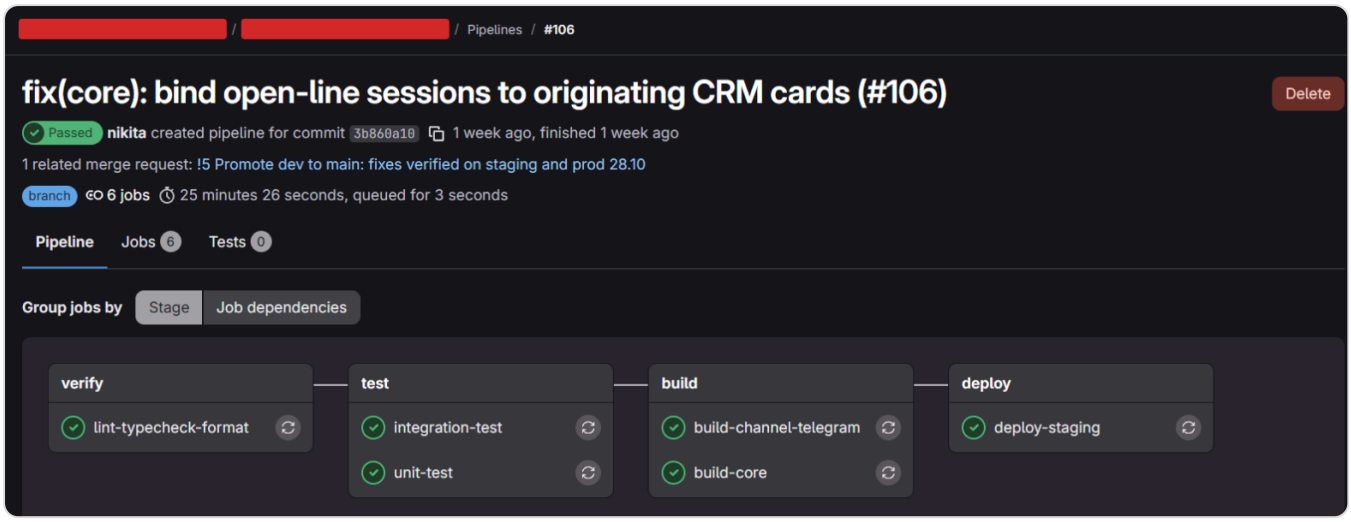
> **A release pipeline for a team service**
>
> For an internal service I made a GitLab pipeline: code checks, unit and integration tests, build, and release to a staging server. One real run took 25 minutes and passed all 6 jobs.

> **Updates that do not interrupt users (my own public project)**
>
> A deploy waits until no users are online. A backup runs before each deploy. If the backup fails, the deploy stops. I tested a full restore on the live server. User data stayed safe.

How the result looks



The bot protection lives in code, not in my head. Real screenshot from my work. Client names are covered in red.



Real GitLab pipeline from my work: checks, tests, build, release to staging. The Premium package adds a similar pipeline for your app. Client names are covered in red.

What you get in this gig

- Basic: I launch your ready Docker app on one server with HTTPS and give you notes.
- Standard: I also prepare the Docker files for your working app.
- Premium: I also add one pipeline. It builds and releases your updates by one button.
- In all packages, your database stays closed to the internet.

Words in this document

Docker	a tool that packs an app with all it needs, so it runs the same on any server.
Server	a computer on the internet that runs your app.
HTTPS	a secure connection. The browser shows a lock icon.
Pipeline	an automatic sequence of steps that tests and releases a new version.

I do not show client names, server addresses or private data. I share them only with the client's permission. Icons: Tabler Icons, MIT license.